



SAVITRIBAI PHULE PUNE UNIVERSITY

Mobile Computing

COMPUTER SCIENCE
Fourth Year (2019 Pattern)

Unit-1 (NOTES)

PUNE
UNIVERSITY

UNIT I: Algorithms and Problem Solving

What are Algorithms?

Definition: An algorithm is a finite set of well-defined, step-by-step instructions for a computer to follow to solve a specific problem or perform a computation. It's like a recipe that takes some input, processes it through a series of defined steps, and produces a desired output. The steps must be unambiguous and must terminate after a finite number of steps.

Detailed Explanation:

- **Inputs and Outputs:** An algorithm must accept one or more inputs and produce a single or multiple outputs.
- **Definiteness:** Each instruction must be clear and unambiguous.
- **Finiteness:** The algorithm must terminate after a finite number of steps for all possible inputs.
- **Effectiveness:** Each step must be simple enough to be executed in a reasonable amount of time.

Real-World Examples:

- **A cooking recipe:** The instructions for baking a cake, including ingredients (input), mixing and baking steps (process), and the final cake (output).
- **Directions to a location:** Step-by-step instructions from a navigation app like Google Maps to get from your current location to a destination.
- **A sorting process:** Arranging a deck of cards in ascending order involves a series of defined steps, like finding the smallest card and placing it at the start, repeating the process.

Applications:

- **Search engines:** Algorithms are used to rank web pages based on relevance to a user's query.
- **Social media feeds:** Algorithms determine the order and type of content a user sees on their feed.

- **E-commerce recommendations:** They suggest products to customers based on their browsing and purchase history.

Important Notes / Key Takeaways:

- An algorithm is a conceptual blueprint, not the actual code.
 - The same problem can be solved by multiple different algorithms, each with varying efficiency.
 - Efficiency is often measured by the time and space resources an algorithm uses.
-

Algorithms as Technology

Definition: Algorithms are considered a technology because they are an essential component of modern computing, driving innovation and enabling a wide range of applications. They are not just theoretical concepts but practical tools that process information, solve complex problems, and automate tasks, forming the backbone of software and systems.

Detailed Explanation:

- **The Bridge between Theory and Practice:** An algorithm is the abstract solution, and its implementation in a programming language makes it a practical, usable technology.
- **Enabling Automation:** Algorithms allow for the automation of tasks that would be impossible or highly inefficient for humans to perform manually.
- **Foundation of Innovation:** Many technological breakthroughs, from machine learning to cryptography, are fundamentally built upon sophisticated algorithms.

Real-World Examples:

- **AI and Machine Learning:** The **PageRank algorithm** is what made Google's search engine so effective by ranking web pages based on link popularity.
- **Cryptocurrency:** The **SHA-256 algorithm** is used to secure Bitcoin transactions and create new blocks on the blockchain.
- **Data Compression:** **Huffman coding** is an algorithm that compresses data by assigning shorter codes to more frequent characters, making file transfers faster.

Applications:

- **Medical imaging:** Algorithms analyze MRI and CT scans to detect abnormalities.
- **Financial trading:** High-frequency trading systems use complex algorithms to execute thousands of trades per second.
- **Traffic management:** Algorithms optimize traffic light timings to reduce congestion in cities.

Important Notes / Key Takeaways:

- Algorithms are a form of intellectual property and can be a source of competitive advantage.
 - The evolution of technology is directly tied to the development of more powerful and efficient algorithms.
 - Understanding algorithms is crucial for anyone working in technology and computer science.
-

Evolution of Algorithms

Definition: The evolution of algorithms is a journey from simple, mechanical procedures to highly complex, self-learning systems. Initially, they were designed for mathematical calculations, but with the advent of computers, their scope expanded to handle data processing, communication, and artificial intelligence, constantly adapting to new hardware and computational challenges.

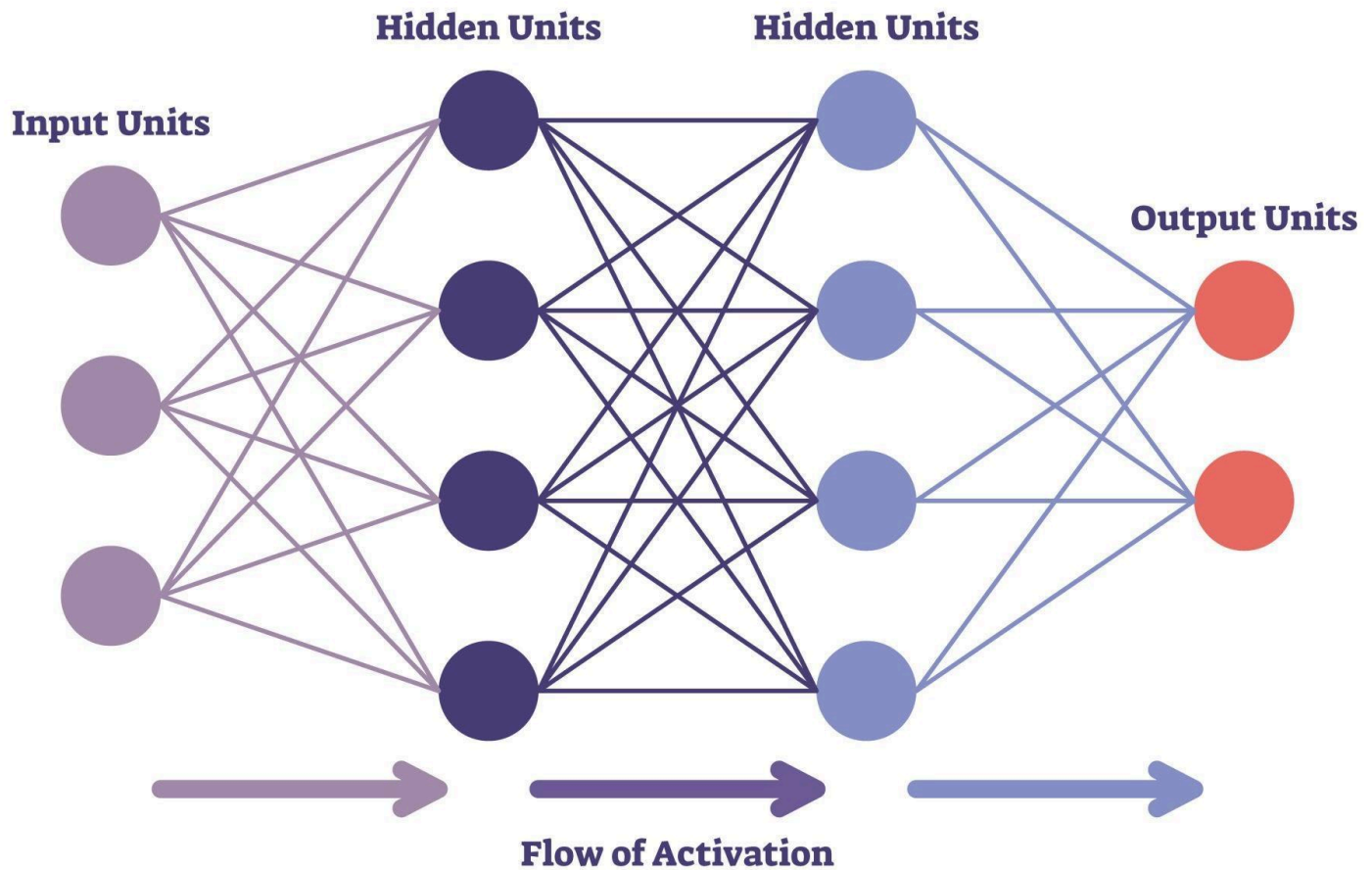
Detailed Explanation:

- **Early Beginnings:** The concept of an algorithm dates back to ancient times with mathematical procedures described by figures like Euclid.
- **The Age of Mechanical Computing:** With early computers like the **Analytical Engine**, algorithms were used for specific arithmetic tasks.
- **The Digital Revolution:** The rise of digital computers led to the formal study of algorithms for sorting, searching, and general data processing.
- **The Modern Era:** The growth of the internet, big data, and machine learning has driven the development of highly complex and specialized algorithms for tasks like pattern recognition and natural language processing.

Real-World Examples:

- **Sorting Algorithms:** From simple **Bubble Sort** (early and inefficient) to highly optimized **Quicksort** and **Merge Sort** (used in modern systems).
- **Searching Algorithms:** The evolution from simple linear search to efficient algorithms like **Binary Search** for sorted data.
- **AI Algorithms:** The progression from **expert systems** (rule-based) to **neural networks** and **deep learning** models, which can learn from data.

AI Neural Networks



Applications:

- **Drug discovery:** New algorithms are being developed to simulate molecular interactions and accelerate the search for new drugs.
- **Self-driving cars:** They rely on advanced algorithms to process sensor data, make decisions, and navigate safely.
- **Climate modeling:** Complex algorithms simulate climate patterns to predict future changes.

Important Notes / Key Takeaways:

- Algorithmic evolution is driven by both theoretical advancements and practical needs.
 - The increasing power of computers has enabled the use of more complex and computationally intensive algorithms.
 - The development of an algorithm is often a feedback loop: new hardware enables new algorithms, which in turn drive the need for better hardware.
-

Design of Algorithm

Definition: Algorithm design is the process of creating a plan or blueprint for an algorithm to solve a specific problem. It involves understanding the problem, identifying the required inputs and outputs, and devising a series of logical steps to transform the inputs into the desired outputs efficiently.

Detailed Explanation:

- **Problem Understanding:** The first step is to fully understand the problem's requirements and constraints.
- **Algorithm Paradigm:** Deciding on a general approach, such as **divide and conquer**, **dynamic programming**, or a **greedy** approach.
- **Data Structures:** Choosing the right data structures (e.g., arrays, linked lists, trees) is crucial for an efficient algorithm.
- **Step-by-Step Logic:** Writing out the logical steps in a clear, unambiguous way, often using **pseudocode** or **flowcharts**.

Real-World Examples:

- **Route Planning:** A GPS app uses a **greedy algorithm** (like **Dijkstra's algorithm**) to find the shortest path by always choosing the next closest node.
- **Scheduling a meeting:** An algorithm to find the optimal time for a meeting among multiple people might use a **backtracking** approach to explore different time slots.
- **Searching for a book in a library:** The **binary search algorithm** is the best design for a sorted list (alphabetical order) of books, repeatedly dividing the list in half.

Applications:

- **Database systems:** Algorithms are designed to optimize database queries for faster data retrieval.
- **Computer graphics:** Algorithms for rendering, shading, and texture mapping are fundamental to video games and animation.
- **Network routing:** Routers use algorithms to determine the most efficient path for data packets to travel across a network.

Important Notes / Key Takeaways:

- A good algorithm design balances correctness with efficiency (time and space complexity).
 - There's often a trade-off between the simplicity of a design and its performance.
 - Thorough testing and analysis are integral parts of the design process.
-

Need of Correctness of Algorithm

Definition: The correctness of an algorithm is the assurance that it will always produce the right output for all valid inputs and will terminate. Without correctness, an algorithm is useless, as it cannot be relied upon to solve the problem it was designed for.

Detailed Explanation:

- **Ensuring Reliability:** A correct algorithm guarantees that the system it powers will behave as expected, preventing errors and failures.
- **Handling All Cases:** A correct algorithm must work for not only typical inputs but also for **edge cases** and **boundary conditions**.

- **Trust and Safety:** In critical applications like medical devices, financial systems, or autonomous vehicles, the correctness of the algorithm is a matter of safety and trust.

Real-World Examples:

- **Banking transactions:** An algorithm to process a withdrawal must correctly subtract the exact amount from the user's account and ensure the balance does not become negative unless an overdraft is allowed.
- **Medical dosage calculations:** An algorithm for calculating a patient's drug dosage must be 100% correct to prevent a medical error.
- **Airline reservation systems:** The algorithm must accurately check seat availability and prevent overbooking, which could lead to major logistical problems.

Applications:

- **Cybersecurity:** Correct cryptographic algorithms are essential for securing data from unauthorized access.
- **Robotics:** A robot's navigation algorithm must be correct to avoid collisions and perform tasks safely.
- **Scientific simulations:** The algorithms used to model climate or chemical reactions must be correct to produce accurate results.

Important Notes / Key Takeaways:

- Correctness is a binary property: an algorithm is either correct or it is not.
- Proving correctness often involves **mathematical induction** or **formal verification** methods.
- Correctness is distinct from efficiency; a correct algorithm can be slow, and an efficient algorithm can be incorrect.

Confirming Correctness of Algorithm – sample examples

Definition: Confirming the correctness of an algorithm involves a systematic process of testing and logical reasoning to prove that it meets its specifications. This can be done through a combination of **walkthroughs**, **testing with diverse inputs**, and **formal proofs**.

Detailed Explanation:

- **Walkthrough (Dry Run):** Step through the algorithm with a simple example on paper to see if it produces the expected output.
- **Test Cases:** Run the algorithm on a wide range of test cases, including typical inputs, edge cases (e.g., empty input, single-element input), and invalid inputs.
- **Formal Proofs:** Use mathematical induction or other formal methods to prove that the algorithm's logic is sound for all possible inputs.

Sample Examples:

- **Algorithm: Linear Search**
 - **Goal:** Find an element x in an unsorted array A .
 - **Logic:** Iterate through the array from the start. If $A[i]$ is equal to x , return the index i . If the loop finishes without finding x , return -1.
 - **Correctness Confirmation:**
 - **Test Case 1 (Typical):** $A = [2, 5, 8, 12, 16]$, $x = 8$. The algorithm correctly returns index 2.
 - **Test Case 2 (Edge Case - First Element):** $A = [2, 5, 8, 12, 16]$, $x = 2$. Correctly returns index 0.
 - **Test Case 3 (Edge Case - Not Found):** $A = [2, 5, 8, 12, 16]$, $x = 10$. The loop completes, and it correctly returns -1.
- **Algorithm: Bubble Sort**
 - **Goal:** Sort an array in ascending order.
 - **Logic:** Repeatedly step through the list, comparing adjacent elements and swapping them if they are in the wrong order.
 - **Correctness Confirmation:**
 - **Test Case 1 (Typical):** $A = [5, 1, 4, 2]$. After several passes, it correctly produces $[1, 2, 4, 5]$.
 - **Test Case 2 (Already Sorted):** $A = [1, 2, 3]$. The algorithm will still run, but no swaps will occur, and it correctly returns the sorted array.

Applications:

- **Software QA:** Quality assurance teams use extensive test cases to verify the correctness of new features.
- **Compiler design:** Compiler developers must ensure that their algorithms correctly translate source code into machine code.

- **Network protocols:** The correctness of protocols like TCP/IP is verified to ensure reliable data transfer.

Important Notes / Key Takeaways:

- Testing can show the presence of errors but not their absence.
 - A comprehensive test suite is crucial for building confidence in an algorithm's correctness.
 - For critical applications, formal proofs are often required in addition to testing.
-

Iterative Algorithm Design Issues

Definition: Iterative algorithm design involves building a solution that repeatedly performs a set of operations until a condition is met. Common issues in this approach include managing state, handling termination conditions, and ensuring correctness throughout the loop.

Detailed Explanation:

- **State Management:** An iterative algorithm often maintains a "state" (variables that change with each iteration). A key issue is ensuring this state is updated correctly in each step.
- **Termination:** The loop must have a clear and guaranteed termination condition. A common error is an **infinite loop**, where the condition is never met.
- **Off-by-One Errors:** A classic mistake in iterative design is using the wrong loop bounds (e.g., $i < n$ instead of $i \leq n$), leading to missing or extra iterations.
- **Maintaining Invariants:** An **invariant** is a property that remains true throughout the loop. Designing and maintaining invariants is a powerful way to ensure correctness.

Real-World Examples:

- **Calculating a sum:** An iterative algorithm to sum the elements of an array.
 - **Issue:** Forgetting to initialize the **sum** variable to 0 could lead to an incorrect result.
- **Finding the factorial:** An iterative algorithm to calculate $n!$.

- **Issue:** Not handling the base case $n = 0$ correctly, or an off-by-one error in the loop that calculates the product.
- **Binary Search:** The iterative version of binary search on a sorted array.
 - **Issue:** Incorrectly updating the **low** or **high** pointers, which can cause the search to fail or enter an infinite loop.

Applications:

- **Data processing:** Iterative algorithms are used in data pipelines to process large datasets in chunks.
- **Game development:** Game loops are a form of iterative algorithm that updates the game state in each frame.
- **Scientific simulations:** Many numerical methods, like those used to solve equations, are iterative in nature.

Important Notes / Key Takeaways:

- **Infinite loops** are a major issue to be aware of.
 - **Loop invariants** are a powerful tool for reasoning about the correctness of iterative algorithms.
 - Careful **initialization** and **termination** conditions are critical for a correct iterative design.
-

Problem Solving Principles

Classification of problem

Definition: Problems in computer science can be classified based on their nature, the type of data they operate on, and the resources required to solve them. This classification helps in choosing the most appropriate algorithmic approach and understanding the inherent difficulty of the problem.

Detailed Explanation:

- **Decision Problems:** A problem where the output is a simple "yes" or "no" answer (e.g., "Is this number prime?").

- **Optimization Problems:** The goal is to find the best possible solution from a set of possible solutions (e.g., "What is the shortest path from A to B?").
- **Search Problems:** Finding a specific element or set of elements that satisfies certain criteria (e.g., "Find all files with the `.jpg` extension.").
- **Counting Problems:** Determining the number of solutions that satisfy a given condition (e.g., "How many ways can I arrange these items?").

Real-World Examples:

- **Decision:** Is a given password strong enough?
- **Optimization:** Finding the most profitable portfolio of stocks.
- **Search:** Finding a specific contact in your phone book.
- **Counting:** Counting the number of unique visitors to a website.

Applications:

- **AI:** Classifying problems helps in designing appropriate machine learning models (e.g., classification models for decision problems, reinforcement learning for optimization).
- **Operations Research:** Optimizing logistics, resource allocation, and supply chains.
- **Database Management:** Designing efficient search and retrieval systems.

Important Notes / Key Takeaways:

- Some problems can fit into multiple categories.
- The classification of a problem often guides the choice of the appropriate algorithm.
- Some problems are inherently harder than others (e.g., NP-hard problems).

Problem Solving Strategies

Definition: Problem-solving strategies are high-level approaches and techniques used to design algorithms. These strategies provide a framework for breaking down a complex problem into smaller, manageable parts and developing a solution.

Detailed Explanation:

- **Brute-Force:** Trying all possible solutions to find the correct one. Simple but often very inefficient.
- **Greedy Approach:** At each step, make the locally optimal choice in the hope of finding a globally optimal solution. This doesn't always work but is fast when it does.
- **Divide and Conquer:** Breaking a problem into smaller subproblems of the same type, solving them independently, and combining their results.
- **Dynamic Programming:** Solving a problem by breaking it down into overlapping subproblems and storing the results of the subproblems to avoid re-computation.
- **Backtracking:** Exploring a solution space by building a solution incrementally and "backtracking" if a path proves to be incorrect.

Real-World Examples:

- **Brute-Force:** Trying every single combination on a padlock to open it.
- **Greedy:** Choosing the biggest bill or coin at each step to give change.
- **Divide and Conquer:** The **Merge Sort** algorithm, which repeatedly divides an array in half until it gets single elements, then merges them back in sorted order.
- **Dynamic Programming:** Finding the shortest path in a network by solving for the shortest path to intermediate nodes first.
- **Backtracking:** Solving a Sudoku puzzle.

Applications:

- **Greedy:** Used in network routing algorithms and scheduling problems.
- **Divide and Conquer:** Found in efficient sorting (Merge Sort) and search (Binary Search) algorithms.
- **Dynamic Programming:** Used in bioinformatics for sequence alignment and in route-planning software.

Important Notes / Key Takeaways:

- Choosing the right strategy is crucial for an efficient solution.
 - Many complex algorithms are a combination of these strategies.
 - Understanding the strengths and weaknesses of each strategy is key to effective algorithm design.
-

Classification of time complexities (linear, logarithmic etc.)

Definition: Time complexity is a measure of the amount of time an algorithm takes to run as a function of the size of its input. It is expressed using **Big O notation** (O), which describes the growth rate of the algorithm's runtime as the input size increases.

Detailed Explanation:

- **$O(1)$ (Constant Time):** The time taken to run is the same regardless of the input size. Example: Accessing an element in an array by its index.
- **$O(\log n)$ (Logarithmic Time):** The time increases slowly as the input size grows. Typically occurs when the algorithm repeatedly divides the problem size in half. Example: **Binary Search**.
- **$O(n)$ (Linear Time):** The time is directly proportional to the size of the input. Example: A simple **linear search** or iterating through an array to find a sum.
- **$O(n^2)$ (Quadratic Time):** The time is proportional to the square of the input size. Often involves nested loops. Example: **Bubble Sort** or **Insertion Sort**.
- **$O(n \log n)$ (Linearithmic Time):** A common complexity for efficient sorting algorithms. Example: **Merge Sort** or **Quicksort**.
- **$O(2^n)$ (Exponential Time):** The time doubles with each addition to the input size. Very inefficient for large inputs. Example: Finding all subsets of a set.

Real-World Examples:

- **$O(1)$:** Getting the top item from a stack.
- **$O(\log n)$:** Searching for a name in a dictionary.
- **$O(n)$:** Reading a list of all names in a directory.
- **$O(n^2)$:** Comparing every team in a tournament with every other team.
- **$O(n \log n)$:** Sorting a large database of records.
- **$O(2^n)$:** Finding the optimal path for a traveling salesman visiting all cities.

Applications:

- **Software Engineering:** Used to predict the performance of an algorithm and choose the most efficient solution for a given problem.
- **System Design:** Designing scalable systems that can handle increasing amounts of data.

- **Competitive Programming:** A fundamental concept for solving algorithmic problems efficiently.

Important Notes / Key Takeaways:

- **Big O notation** describes the **worst-case** scenario.
- It's a measure of **growth rate**, not absolute time.
- **Space complexity** is a related concept that measures the amount of memory an algorithm uses.

Summary Table for Revision

Main Topic Name	Brief Definition	3 Real-World Example
What are Algorithms?	An algorithm is a finite set of clear, step-by-step instructions for a computer to follow to solve a specific problem. It takes an input, processes it, and produces a defined output in a finite number of steps. It's the logical blueprint for a computation.	1. A cooking recipe. 2. Step-by-step directions from a GPS. 3. The process of arranging a deck of cards.
Algorithms as Technology	Algorithms are a form of technology because they are practical, operational tools that process information and automate tasks, forming the fundamental backbone of all software and systems. They bridge theoretical concepts with real-world applications.	1. Google's search engine ranking algorithm. 2. Cryptocurrency security algorithms. 3. Data compression techniques like Huffman coding.
Evolution of Algorithms	The evolution of algorithms tracks their development from simple mathematical procedures to complex, self-learning systems. This progression has been driven by the increasing power of hardware and the need to solve more complex problems in data processing and AI.	1. The transition from inefficient Bubble Sort to modern sorting algorithms like Quicksort. 2. The shift from simple, rule-based AI to complex neural networks. 3. The development of navigation algorithms from basic pathfinding to GPS-based route optimization.

Design of Algorithm	Algorithm design is the process of creating a systematic plan for solving a problem, involving understanding the problem, choosing a strategy (like divide and conquer), and defining the logical steps. It is a creative and analytical process.	1. A GPS using Dijkstra's algorithm to find the shortest route. 2. A program using binary search to find a book in a library's sorted catalog. 3. A meeting scheduler using backtracking to find an open time slot for everyone.
Need of Correctness of Algorithm	Correctness is the guarantee that an algorithm will produce the right output for all valid inputs and will always terminate. This is crucial for reliability, safety, and trust in any system, especially in critical applications like finance or medicine.	1. A banking algorithm must correctly subtract money from an account. 2. A medical device's algorithm must accurately calculate a drug dosage. 3. An airline reservation algorithm must prevent overbooking.
Confirming Correctness of Algorithm	This involves a systematic process to verify that an algorithm works as intended. Methods include testing with diverse inputs, performing a walkthrough (dry run), and using formal mathematical proofs to guarantee correctness.	1. Testing a linear search algorithm on an empty array. 2. Verifying a sorting algorithm with an already sorted array. 3. Using test cases to check for off-by-one errors in a loop.
Iterative Algorithm Design Issues	Iterative design uses loops to solve problems, but common issues include managing the changing state of variables, ensuring the loop always terminates, and avoiding errors like off-by-one mistakes that can lead to incorrect results or infinite loops.	1. Forgetting to initialize a sum variable to zero before a loop. 2. A while loop with a condition that is never met, causing an infinite loop. 3. A search loop that incorrectly excludes the last element of an array.
Classification of Problem	Problems in computing are classified based on their nature, such as whether they require a simple yes/no answer (decision), an optimal solution (optimization), or finding a specific item (search). This helps in choosing an appropriate solution strategy.	1. Decision: Is a given number a prime number? 2. Optimization: What is the most efficient delivery route? 3. Search: Finding a specific document on your computer.

Problem Solving Strategies	These are high-level approaches for designing algorithms, such as breaking a problem into smaller parts (divide and conquer) or making the best choice at each step (greedy). The strategy chosen impacts the algorithm's efficiency and correctness.	1. Greedy: An algorithm to provide the least number of coins for change. 2. Divide and Conquer: The Merge Sort algorithm for sorting. 3. Dynamic Programming: An algorithm to calculate the shortest path in a graph.
Classification of Time Complexities	Time complexity measures how an algorithm's runtime scales with input size using Big O notation. It allows us to compare and predict the efficiency of different algorithms, categorizing them as linear, logarithmic, or quadratic, among others.	1. Logarithmic: Searching for a word in a dictionary. 2. Linear: Scanning a document for a specific phrase. 3. Quadratic: A simple sorting algorithm that compares every element with every other element.

Thank-You

We sincerely appreciate your time and attention in reviewing this document. Your feedback and support mean a lot to us, and we look forward to building something valuable together.



Visit site to access further Notes!!!

✨ Why Choose Us?

- 📖 Free & Updated Notes for All Subjects
- 🎯 Exam-Focused Content
- 🔍 Easy Navigation by Unit & Topic
- 💻 Works on Mobile & Desktop

Branches: **CS | IT | AI&DS | AI&ML**



Notes By: puneuniversity.netlify.app

🔗 Visit us: <https://cutt.ly/lrHFgvYl>